Terraform Loops: Mastering Count and For_Each Simply

Copy-pasting Terraform blocks wastes time and makes your infrastructure code fragile. Learning count and for_each allows you to deploy scalable, dynamic cloud resources with just a few clean lines.

What you'll learn:

1. Creating Identical Resources with Count
2. Toggle Resources with Conditional Count
3. Using For_Each with Key-Value Maps
4. Using For_Each with toset() on Lists

Aman Raj Singh

Cloud Operations Engineer @ iManage

linkedin.com/in/mramanrajsingh

1

TIP 1 OF 4

Creating Identical Resources with Count

The count argument accepts a whole number and duplicates the resource block that many times. Inside the block, you can use count.index to get the current loop index starting at 0. This is best for identical resources where individual items don't have distinct configurations or custom IDs.

```
resource "aws_instance" "server" {
  count         = 3
  ami           = "ami-0c55b159cbfafe1f0"
  instance_type = "t3.micro"
  tags = {
    Name = "worker-${count.index + 1}"
  }
}
```

Swipe to tip 2 >> linkedin.com/in/mramanrajsingh

2

TIP 2 OF 4

Toggle Resources with Conditional Count

You can turn any resource on or off by combining count with a ternary condition. If the condition evaluates to true, count is set to 1 and the resource is created. If false, count is 0 and Terraform skips creating it entirely without throwing errors.

```
variable "enable_monitoring" {
  type    = bool
  default = false
}

resource "aws_cloudwatch_metric_alarm" "cpu" {
  count      = var.enable_monitoring ? 1 : 0
  alarm_name = "high-cpu-alarm"
}
```

Terraform for_each & count: Stop Copy-Pasting Resource Blocks

Thursday, November 05, 2026

Swipe to tip 3 >> linkedin.com/in/mramanrajsingh

3

TIP 3 OF 4

Using For_Each with Key-Value Maps

Unlike count, for_each iterates over maps and tracks each resource by its key rather than an index position. Inside the block, each.key returns the map key and each.value returns the assigned value. This prevents accidental recreation when items are added or removed.

```
resource "aws_instance" "env" {
  for_each = {
    dev  = "t3.micro"
    prod = "t3.large"
  }
  ami           = "ami-0c55b159cbfafe1f0"
  instance_type = each.value
  tags = { Environment = each.key }
}
```

Swipe to tip 4 >> linkedin.com/in/mramanrajsingh

4

TIP 4 OF 4

Using For_Each with toset() on Lists

The for_each meta-argument only accepts maps or sets of strings, not plain lists. Wrapping your string list with toset() converts it instantly and ensures all entries are unique. Terraform tracks each resource by string value, making deletes completely safe.

```
locals {
  bucket_names = ["logs", "assets", "reports"]
}

resource "aws_s3_bucket" "app_buckets" {
  for_each = toset(local.bucket_names)
  bucket   = "mycompany-${each.key}-bucket"
}
```

See Key Takeaways on next page >> linkedin.com/in/mramanrajsingh

Key Takeaways

+ Use count primarily for true duplicates or 0/1 condition toggles.
+ Use for_each when items need custom names, sizes, or configurations.
+ Always convert string lists using toset() before passing them to for_each.
+ Reference for_each resources using bracket notation: resource.name["key"].
+ Run terraform plan before applying to review resource address changes.

Watch Out For

! Removing an item from the middle of a count list causes Terraform to destroy and rebuild all subsequent items.
! Passing a regular list directly to for_each without toset() triggers a syntax error.
! Using count when resources require persistent, independent identities across deploys.

Follow for daily Cloud & DevOps guides!

linkedin.com/in/mramanrajsingh